

Oracle's Commands

1. DDL (Data Definition Language) Commands

CREATE TABLE

-- Create a simple employees table

```
CREATE TABLE employees (  
    employee_id NUMBER(6) PRIMARY  
    KEY,  
    first_name VARCHAR2(50),  
    last_name VARCHAR2(50) NOT NULL,  
    email VARCHAR2(100) UNIQUE,  
    hire_date DATE DEFAULT SYSDATE,  
    salary NUMBER(8,2),  
    department_id NUMBER(4)  
);
```

-- Create a departments table

```
CREATE TABLE departments (  
    department_id NUMBER(4)  
    PRIMARY KEY,  
    department_name VARCHAR2(100)  
    NOT NULL,  
    location VARCHAR2(100)  
);
```

ALTER TABLE

-- Add a new column

```
ALTER TABLE employees ADD (phone_number VARCHAR2(20));
```

-- Modify existing column

```
ALTER TABLE employees MODIFY (salary NUMBER(10,2));
```

-- Rename a column

```
ALTER TABLE employees RENAME COLUMN phone_number TO mobile_number;
```

-- Add a constraint

```
ALTER TABLE employees ADD CONSTRAINT fk_dept  
    FOREIGN KEY (department_id) REFERENCES departments(department_id);
```

DROP TABLE

-- Drop a table (be careful!)

DROP TABLE employees;

-- This will fail if there are dependencies, use CASCADE CONSTRAINTS if needed

DROP TABLE employees CASCADE CONSTRAINTS;

TRUNCATE TABLE

-- Remove all data but keep table structure

```
TRUNCATE TABLE employees;
```

RENAME TABLE

-- Rename a table

RENAME employees TO staff;

2. DML (Data Manipulation Language) Commands

INSERT

-- Insert single row

```
INSERT INTO departments (department_id, department_name, location)
VALUES (10, 'IT', 'New York');
```

-- Insert multiple rows

```
INSERT ALL
  INTO departments VALUES (20, 'HR', 'London')
  INTO departments VALUES (30, 'Finance', 'Tokyo')
SELECT * FROM dual;
```

-- Insert with default values

```
INSERT INTO employees (employee_id, first_name, last_name, email, salary, department_id)
VALUES (101, 'John', 'Doe', 'john.doe@email.com', 50000, 10);
```

SELECT

-- Basic SELECT

```
SELECT * FROM departments;
```

-- Select specific columns

```
SELECT first_name, last_name, salary FROM employees;
```

-- Select with alias

```
SELECT first_name || ' ' || last_name AS full_name, salary *  
12 AS annual_salary
```

```
FROM employees;
```

-- Select with WHERE clause

```
SELECT * FROM employees
```

```
WHERE department_id = 10 AND salary > 40000;
```

-- Select with ORDER BY

```
SELECT * FROM employees
```

```
ORDER BY salary DESC, last_name ASC;
```

-- Select with GROUP BY

```
SELECT department_id, COUNT(*) AS  
employee_count, AVG(salary) AS  
avg_salary
```

```
FROM employees
```

```
GROUP BY department_id
```

```
HAVING COUNT(*) > 5;
```

UPDATE

-- Update single column

```
UPDATE employees  
SET salary = 55000  
WHERE employee_id = 101;
```

-- Update multiple columns

```
UPDATE employees  
SET salary = salary * 1.1,  
    last_updated = SYSDATE  
WHERE department_id = 10;
```

-- Update using subquery

```
UPDATE employees  
SET department_id = (  
    SELECT department_id  
    FROM departments  
    WHERE department_name = 'IT'  
)  
WHERE employee_id = 101;
```

DELETE

-- Delete specific rows

```
DELETE FROM employees  
WHERE employee_id = 101;
```

-- Delete all rows (but keep table)

```
DELETE FROM employees;
```

-- Delete with subquery

```
DELETE FROM employees  
WHERE department_id IN (  
    SELECT department_id  
    FROM departments  
    WHERE location = 'New York'  
);
```

MERGE (UPSERT)

-- Insert or Update

MERGE INTO employees e

USING (SELECT 101 as id, 'John' as fname, 'Doe' as lname FROM dual) s

ON (e.employee_id = s.id)

WHEN MATCHED THEN

 UPDATE SET e.first_name = s.fname, e.last_name = s.lname

WHEN NOT MATCHED THEN

 INSERT (employee_id, first_name, last_name)

 VALUES (s.id, s.fname, s.lname);

3. TCL (Transaction Control Language) Commands

COMMIT

-- Start a transaction and save changes permanently

```
INSERT INTO departments VALUES (40, 'Marketing', 'Chicago');
```

```
COMMIT;
```

-- Changes are now permanent

ROLLBACK

-- Undo changes since last commit

INSERT INTO departments VALUES (50, 'Sales', 'Boston');

SELECT * FROM departments; -- Shows the new row

ROLLBACK;

SELECT * FROM departments; -- New row is gone

SAVEPOINT

-- Create savepoints for partial rollbacks

```
INSERT INTO departments VALUES (60, 'R&D', 'Seattle');
```

```
SAVEPOINT sp1;
```

```
INSERT INTO departments VALUES (70, 'Support', 'Miami');
```

```
SAVEPOINT sp2;
```

-- Oops, wrong entry, rollback to sp1

```
ROLLBACK TO SAVEPOINT sp1;
```

```
COMMIT;
```

-- Only department 60 is saved

4. DCL (Data Control Language) Commands

GRANT

-- Grant privileges

```
GRANT SELECT, INSERT ON employees TO scott;
```

-- Grant all privileges

```
GRANT ALL ON departments TO hr_user;
```

-- Grant with grant option

```
GRANT SELECT ON employees TO manager WITH GRANT OPTION;
```

REVOKE

-- Revoke privileges

```
REVOKE INSERT ON employees FROM scott;
```

-- Revoke all privileges

```
REVOKE ALL ON departments FROM hr_user;
```

5. *Advanced Queries and Functions*

Joins

-- INNER JOIN

```
SELECT e.first_name, e.last_name, d.department_name  
FROM employees e  
  
INNER JOIN departments d ON e.department_id =  
d.department_id;
```

-- LEFT JOIN

```
SELECT e.first_name, d.department_name  
FROM employees e  
  
LEFT JOIN departments d ON e.department_id =  
d.department_id;
```

-- Self Join

```
SELECT e1.first_name AS employee,  
e2.first_name AS manager  
FROM employees e1  
  
LEFT JOIN employees e2 ON  
e1.manager_id = e2.employee_id;
```

Aggregate Functions

-- Various aggregate functions

SELECT

 COUNT(*) AS total_employees,

 AVG(salary) AS average_salary,

 MAX(salary) AS highest_salary,

 MIN(salary) AS lowest_salary,

 SUM(salary) AS total_salary

FROM employees;

Date Functions

-- Oracle date functions

SELECT

SYSDATE AS current_date,

EXTRACT(YEAR FROM hire_date) AS hire_year,

TO_CHAR(hire_date, 'MM/DD/YYYY') AS formatted_date,

ADD_MONTHS(hire_date, 6) AS review_date,

MONTHS_BETWEEN(SYSDATE, hire_date) AS months_employed

FROM employees;

String Functions

-- String manipulation

SELECT

 UPPER(first_name) AS upper_name,

 LOWER(last_name) AS lower_name,

 INITCAP(email) AS proper_case,

 SUBSTR(first_name, 1, 3) AS name_prefix,

 LENGTH(last_name) AS name_length,

 TRIM(' ' FROM ' text ') AS trimmed_text

FROM employees;

Subqueries

-- Subquery in WHERE

```
SELECT first_name, last_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

-- Subquery in SELECT

```
SELECT
    e.first_name,
    e.salary,
    (SELECT AVG(salary) FROM employees) AS
company_avg
FROM employees e;
```

-- Correlated subquery

```
SELECT department_id, employee_id,
salary
FROM employees e1
WHERE salary > (
    SELECT AVG(salary)
    FROM employees e2
    WHERE e2.department_id =
e1.department_id
);
```

Views

-- Create a view

```
CREATE VIEW high_earners AS
```

```
SELECT employee_id, first_name, last_name, salary
```

```
FROM employees
```

```
WHERE salary > 70000;
```

-- Query the view

```
SELECT * FROM high_earners;
```

-- Drop view

```
DROP VIEW high_earners;
```

Sequences

-- Create sequence for auto-increment

```
CREATE SEQUENCE emp_seq  
  START WITH 1000  
  INCREMENT BY 1  
  MAXVALUE 999999  
  NOCACHE  
  NOCYCLE;
```

-- Use sequence

```
INSERT INTO employees (employee_id,  
  first_name, last_name)  
VALUES (emp_seq.NEXTVAL, 'Jane',  
  'Smith');
```

-- Get current sequence value

```
SELECT emp_seq.CURRVAL FROM dual;
```

Indexes

-- Create index

```
CREATE INDEX idx_emp_lastname ON employees(last_name);
```

-- Create unique index

```
CREATE UNIQUE INDEX idx_emp_email ON employees(email);
```

-- Drop index

```
DROP INDEX idx_emp_lastname;
```

6. Useful Oracle-Specific Features

Dual Table

-- DUAL is a dummy table for calculations

SELECT

2 + 2 AS sum,

SYSDATE AS today,

USER AS current_user,

'Hello World' AS greeting

FROM dual;

Pseudo Columns

-- ROWNUM - limits rows

```
SELECT * FROM employees
```

```
WHERE ROWNUM <= 5;
```

-- ROWID - physical location

```
SELECT ROWID, employee_id, first_name
```

```
FROM employees;
```

Analytical Functions

-- ROW_NUMBER() - assign sequential numbers

SELECT

employee_id,

first_name,

salary,

ROW_NUMBER() OVER (ORDER BY salary
DESC) AS salary_rank

FROM employees;

-- LAG/LEAD - access previous/next rows

SELECT

employee_id,

salary,

LAG(salary, 1) OVER (ORDER BY
employee_id) AS previous_salary,

LEAD(salary, 1) OVER (ORDER BY
employee_id) AS next_salary

FROM employees;

Thank you!